

QUESTION BANK ANSWERS CP:

1. What is Data Structures? What are the basic operations of data structure?

Definition of Data Structures

A **Data Structure** is a way of organizing, storing, and managing data efficiently so that it can be accessed and manipulated effectively. It provides a systematic way to manage data and perform operations like insertion, deletion, searching, and sorting.

Examples: Arrays, Linked Lists, Stacks, Queues, Trees, Graphs, and Hash Tables.

Basic Operations on Data Structures

The basic operations on data structures include:

1. **Traversal:** Accessing and printing elements one by one.
2. **Insertion:** Adding a new element to the data structure.
3. **Deletion:** Removing an element from the data structure.
4. **Searching:** Finding an element in the data structure.
5. **Sorting:** Arranging elements in ascending or descending order.
6. **Updating:** Changing the value of an element.

2. Write down the basic terminologies used in data structure.

In **Data Structures**, the following terminologies are commonly used:

1. Data

- **Definition:** Data refers to raw information stored in memory.
 - **Example:** Numbers, Characters, Strings, or any form of data like 45, "John", 78.9.
-

2. Data Structure

- **Definition:** A Data Structure is a collection of data organized in a particular way for efficient processing.
 - **Example:** Arrays, Linked Lists, Stacks, Queues, Trees, and Graphs.
-

3. Element

- **Definition:** Each piece of data in a data structure is called an **element**.
 - **Example:** In an array {10, 20, 30}, each number is an element.
-

4. Node

- **Definition:** A **Node** is a fundamental part of linked structures like Linked Lists and Trees. It contains data and pointers (references) to other nodes.

- **Example:**

[10 | next] → [20 | next] → [30 | NULL]

5. Pointer

- **Definition:** A pointer is a variable that stores the memory address of another variable. It is heavily used in dynamic data structures.
- **Example in C:**

```
int a = 10;
```

```
int *ptr = &a; // Pointer storing address of variable a
```

6. Linear Data Structure

- **Definition:** A data structure where elements are arranged in a sequential manner.
 - **Examples:** Arrays, Linked Lists, Stacks, and Queues.
-

7. Non-Linear Data Structure

- **Definition:** A data structure where elements are connected in a hierarchical manner.
 - **Examples:** Trees, Graphs.
-

8. Abstract Data Type (ADT)

- **Definition:** ADT defines the operations that can be performed on a data structure without specifying its implementation.
 - **Examples:** Stack (with operations like Push, Pop, and Peek), Queue, and List.
-

9. Algorithm

- **Definition:** A step-by-step procedure to solve a problem in a finite number of steps.
 - **Example:** Sorting algorithms like Bubble Sort and Quick Sort.
-

10. Complexity

- **Definition:** It refers to the performance of an algorithm in terms of time and space.
 - **Time Complexity:** Amount of time an algorithm takes to run.
 - **Space Complexity:** Amount of memory used by the algorithm.
 - **Example:** Sorting an array has a time complexity of $O(n^2)$ using Bubble Sort.
-

11. Stack

- **Definition:** A stack is a linear data structure that follows the **LIFO (Last In First Out)** principle.
- **Example:** Undo operation in a text editor.

12. Queue

- **Definition:** A queue is a linear data structure that follows the **FIFO (First In First Out)** principle.
 - **Example:** A line of people waiting at a ticket counter.
-

13. Linked List

- **Definition:** A linear data structure where elements (nodes) are connected using pointers.
- **Example:**

10 → 20 → 30 → NULL

14. Tree

- **Definition:** A non-linear data structure consisting of nodes where one node acts as the root and other nodes are connected as children.
 - **Example:** A family tree or a file directory.
-

15. Graph

- **Definition:** A graph consists of nodes connected by edges. It is used to represent networks.
 - **Example:** Social network graphs like Facebook.
-

3. Define Abstract Data Type (ADT) and explain its fundamental concepts.

Definition of Abstract Data Type (ADT)

An **Abstract Data Type (ADT)** is a model for a data structure that specifies:

- The type of data stored.
- The operations that can be performed on the data.
- The behavior of these operations.

It **does not** specify how the operations will be implemented. ADTs focus on **what** operations are required, not **how** they will be implemented.

Fundamental Concepts of ADT

1. Data

- The values or information stored in the data structure.

2. Operations

- A set of functions to manipulate the data, like inserting, deleting, and accessing elements.

3. Implementation Independence

- ADTs hide the implementation details. Users only interact with the data structure through its defined operations.

4. Encapsulation

- The internal representation of the data is hidden, ensuring data integrity.

5. Abstraction

- Users can use the data structure without knowing its internal working.

Examples of ADT

- **List ADT:** Supports operations like insert, delete, traverse, and search.
- **Stack ADT:** Supports operations like push, pop, and peek.
- **Queue ADT:** Supports operations like enqueue, dequeue, and isEmpty.
- **Deque ADT:** Supports insertion and deletion from both ends.
- **Set ADT:** Supports union, intersection, and difference operations.

Question 4 ans

```
#include <stdio.h>
```

```
#define MAX 500
```

```
// Define the Array as an Abstract Data Type
```

```
struct Array {  
    int arr[MAX];  
    int capacity;  
    int lastIndex;  
};
```

```
// Initialization Function
```

```
void init(struct Array *a, int size) {  
    if (size > MAX) {  
        printf("Error: Size exceeds maximum limit of %d.\n", MAX);  
        return;  
    }  
    a->capacity = size;  
    a->lastIndex = -1; // Array is empty initially  
    printf("Array initialized with capacity %d.\n", size);  
}
```

```
int main() {
```

```
    struct Array a;  
    int size;
```

```
    printf("Enter the size of the array: ");
```

```

scanf("%d", &size);
init(&a, size);

return 0;
}

```

Question 5 ans

Data structures are primarily classified into **two categories**:

1. **Linear Data Structures**
2. **Non-Linear Data Structures**

Criteria	Linear Data Structures	Non-Linear Data Structures
Data Arrangement	Sequential (one after another)	Hierarchical or interconnected
Memory Usage	Can use contiguous or non-contiguous memory	Mostly non-contiguous memory
Traversal	Simple traversal using loops or pointers	Requires complex algorithms like DFS or BFS
Implementation	Easier to implement using arrays or linked lists	Harder to implement due to multiple pointers
Complexity	Generally less complex	More complex with higher computational needs
Applications	Suitable for simple data management	Best for complex relationships like file systems
Examples	Array, Stack, Queue, Linked List	Tree, Graph, Heap

6. Consider a zero-indexed array A with dimensions 5 x 6 x 7 that stores 4-byte integers located at address 6400. Calculate the address of A(4, 5, 6) using row-major order and column-major order.

Step 1: Understanding the Problem

- **Array Dimensions:** $5 \times 6 \times 7$
 - **5:** Number of rows (first dimension)
 - **6:** Number of columns (second dimension)
 - **7:** Depth or layers (third dimension)
- **Base Address:** 6400
- **Element to Find:** A(4,5,6)
- **Element Size:** 4 bytes (Integer)

Step 2: Formula for Address Calculation

(A) Row-Major Order (C-style)

Formula:

$$\text{Address} = \text{Base Address} + (i \cdot (n_2 \cdot n_3) + j \cdot n_3 + k) \cdot \text{Element Size}$$

Where:

- $i, j, k, i_1, j_1, k_1 \rightarrow$ Index of element $A(i, j, k)$
- $n_1, n_2, n_3 \rightarrow$ Dimensions of the array
- Base Address = 6400
- Element Size = 4 bytes

Given:

- $i=4, j=5, k=6$
- $n_1=5, n_2=6, n_3=7$

Given:

- $i = 4, j = 5, k = 6$
- $n_1 = 5, n_2 = 6, n_3 = 7$

Substituting the values:

$$\begin{aligned}\text{Address} &= 6400 + (4 \cdot (6 \cdot 7) + 5 \cdot 7 + 6) \cdot 4 \\ &= 6400 + (4 \cdot 42 + 5 \cdot 7 + 6) \cdot 4 \\ &= 6400 + (168 + 35 + 6) \cdot 4 \\ &= 6400 + 209 \cdot 4 \\ &= 6400 + 836 = 7236\end{aligned}$$

Final Address (Row-Major Order): 7236



(B) Column-Major Order (Fortran-style)

Formula:

$$\text{Address} = \text{Base Address} + (k \cdot (n_1 \cdot n_2) + j \cdot n_1 + i) \cdot \text{Element Size}$$

Substituting the values:

$$\begin{aligned}\text{Address} &= 6400 + (6 \cdot (5 \cdot 6) + 5 \cdot 5 + 4) \cdot 4 \\ &= 6400 + (6 \cdot 30 + 5 \cdot 5 + 4) \cdot 4 \\ &= 6400 + (180 + 25 + 4) \cdot 4 \\ &= 6400 + 209 \cdot 4 \\ &= 6400 + 836 = 7236\end{aligned}$$

Final Address (Column-Major Order): 7236

Question 7 ans

```
int isFull(struct Array *a) {
    if(a->lastIndex == a->capacity - 1)
        return 1;
    else
        return 0;
}
```

Question 8 ans

```
void create(struct Array *a) {
    int ele = getInput("Enter any number (999 to stop): ");
    while (ele != 999 && isFull(a) != 1) {
        if (Add(a, ele) == -1) break;
        ele = getInput("Enter any number (999 to stop): ");
    }
}
```

question 9 ans

```
void delete(struct Array *a, int index) {
    if (index <= a->lastIndex) {
        for (int i = index; i < a->lastIndex; i++) {
            a->arr[i] = a->arr[i + 1];
        }
        a->lastIndex--;
    }
}
```

```

        printf("\nDelete operation successful\n");
    }
}

```

question 10 ans

```

void addAtRandomIndex(struct Array *a, int index, int elem) {
    int i;
    if (index <= a->lastIndex && index >= 0) {
        for (i = a->lastIndex; i >= index; i--) {
            a->arr[i + 1] = a->arr[i];
        }
        a->arr[index] = elem;
        a->lastIndex++;
    }
}

```

11. Write an array reversal algorithm using in-place shifting operation.

C Algorithm for In-Place Array Reversal

1. Start
2. Input the array arr and its size n.
3. Initialize two pointers:
 - left = 0 → Points to the first element.
 - right = n-1 → Points to the last element.
4. Repeat the following steps until left < right:
 - Swap the elements at left and right.
 - Increment left by 1.
 - Decrement right by 1.
5. Print the reversed array.
6. End

12. What happens if the exchange process continues for n steps rather than n/2 steps in array reversal algorithm?

- The reversal algorithm performs only **n/2 swaps** since each swap reverses **two elements**.
- If the exchange continues for n steps:
 - The array will reverse twice, returning to its original state.
 - This is redundant and inefficient.

13. Write an algorithm that places k-th element of an array in position 1, the (k+1)th in position 2, and so on.

- **C Algorithm for Circular Rearrangement**

- ```
#include <stdio.h>
```
- 
- ```
void rearrange(int arr[], int n, int k) {
```
- ```
 int temp[n];
```
- ```
    for (int i = 0; i < n; i++) {
```
- ```
 temp[i] = arr[(k + i) % n];
```
- ```
    }
```
- ```
 for (int i = 0; i < n; i++) {
```
- ```
        arr[i] = temp[i];
```
- ```
 }
```
- ```
}
```
-
- ```
void display(int arr[], int n) {
```
- ```
    for (int i = 0; i < n; i++) {
```
- ```
 printf("%d ", arr[i]);
```
- ```
    }
```
- ```
 printf("\n");
```
- ```
}
```
-
- ```
int main() {
```
- ```
    int arr[] = {10, 20, 30, 40, 50};
```
- ```
 int n = 5, k = 2;
```
- ```
    printf("Original Array: ");
```
- ```
 display(arr, n);
```
- 
- ```
    rearrange(arr, n, k);
```
- ```
 printf("Rearranged Array: ");
```
- ```
    display(arr, n);
```
-
- ```
 return 0;
```
- ```
}
```
-

14. Write an algorithm that rearranges the elements of an array so that all those originally stored at odd suffixes are placed before those at even suffixes.

Algorithm

1. **Start**
2. Input the array and its size, n.
3. Create two temporary arrays:
 - oddArray to store elements at odd indices.
 - evenArray to store elements at even indices.

4. Traverse the input array:
 - If the index is **odd**, store the element in oddArray.
 - If the index is **even**, store the element in evenArray.
5. Copy all elements from oddArray back to the original array.
6. Append all elements from evenArray to the original array.
7. Print the rearranged array.
8. **End**

15. Calculate the address of A[10][20][30] in row-major and column-major order.

Given Data:

- 3D Array: **A[90][30][40]**
- Base Address = **1000**
- Element Size = **2 bytes**
- Starting Index = **A[2][2][2]**

(A) Row-Major Order (C-Style)

Formula:

$$\text{Address} = \text{Base Address} + ((i - l_1) \cdot n_2 \cdot n_3 + (j - l_2) \cdot n_3 + (k - l_3)) \cdot \text{Element Size}$$

Where:

- $i=10, j=20, k=30$
- Lower bounds: $l_1=2, l_2=2, l_3=2$
- Dimensions: $n_1=90, n_2=30, n_3=40$

Calculation:

$$\begin{aligned}
 \text{Address} &= 1000 + ((10 - 2) \cdot 30 \cdot 40 + (20 - 2) \cdot 40 + (30 - 2)) \cdot 2 \\
 &= 1000 + (8 \cdot 1200 + 18 \cdot 40 + 28) \cdot 2 \\
 &= 1000 + (9600 + 720 + 28) \cdot 2 \\
 &= 1000 + 10348 \cdot 2 \\
 &= 1000 + 20696 = 21696
 \end{aligned}$$

Final Address (Row-Major Order): 21696

(B) Column-Major Order (Fortran-Style)

Formula:

$$\text{Address} = \text{Base Address} + ((k - l_3) \cdot n_1 \cdot n_2 + (j - l_2) \cdot n_1 + (i - l_1)) \cdot \text{Element Size}$$

Calculation:

Calculation:

$$\begin{aligned}\text{Address} &= 1000 + ((30 - 2) \cdot 90 \cdot 30 + (20 - 2) \cdot 90 + (10 - 2)) \cdot 2 \\ &= 1000 + (28 \cdot 2700 + 18 \cdot 90 + 8) \cdot 2 \\ &= 1000 + (75600 + 1620 + 8) \cdot 2 \\ &= 1000 + 77228 \cdot 2 \\ &= 1000 + 154456 = 155456\end{aligned}$$

Final Address (Column-Major Order): 155456

16. Write a pseudo code to count the number of students who achieved each possible mark in the counter array.

Pseudo Code for Counting Marks

Algorithm CountMarks(marks, n):

1. Create an array 'count' of size 101 (to cover 0 to 100 marks) and initialize all elements to 0
for i from 0 to 100:
 count[i] $\leftarrow$ 0
2. Traverse the marks array:
for i from 0 to n-1:
 count[marks[i]] $\leftarrow$ count[marks[i]] + 1
3. Print the results:
for i from 0 to 100:
 if count[i] > 0:
 Print "Mark", i, "->", count[i], "students"

17. Modify the algorithm above to obtain a histogram for each ten percentile range (e.g., 0-10%, 11-20%, ...) rather than for each individual mark.

Pseudo Code for Histogram Generation by Percentile Range

Algorithm GenerateHistogram(marks, n):

1. Create an array 'histogram' of size 10 to store student counts for each percentile range.
for i from 0 to 9:
 histogram[i] $\leftarrow$ 0
2. Traverse the marks array:
for i from 0 to n-1:
 index $\leftarrow$ marks[i] / 10

histogram[index] $\leftarrow$ histogram[index] + 1

3. Print the histogram:

for i from 0 to 9:

Print i*10, "-", i*10+9, ":", histogram[i], "students"

18. Write an algorithm to find the number of times the maximum occurs in an array of n elements using only one pass.

Algorithm

1. **Start**
2. Input the array arr and its size n.
3. Initialize two variables:
 - maxVal = First element of the array.
 - count = 1 (since the first element is considered the max initially).
4. Traverse the array from the second element to the last:
 - **If** the current element is greater than maxVal:
 - Update maxVal to the current element.
 - Reset count to 1.
 - **Else If** the current element is equal to maxVal:
 - Increment count by 1.
5. After the traversal, print count as the number of times the maximum occurs.
6. **End**

19. Write a program using function to find the maximum element in an array along with its position.

C Program to Find Maximum and Its Position

```
#include <stdio.h>
```

```
void findMaxAndPosition(int arr[], int n) {
```

```
    int max = arr[0];
```

```
    int pos = 0;
```

```
    for (int i = 1; i < n; i++) {
```

```
        if (arr[i] > max) {
```

```
            max = arr[i];
```

```
            pos = i;
```

```

    }
}
printf("Maximum Element: %d at Position: %d\n", max, pos);
}

int main() {
    int arr[] = {10, 25, 30, 15, 30};
    int n = sizeof(arr) / sizeof(arr[0]);
    findMaxAndPosition(arr, n);
    return 0;
}

```

20. Write a program using function to find the minimum element in an array along with its position.

C Program to Find Minimum and Its Position

```
#include <stdio.h>
```

```

void findMinAndPosition(int arr[], int n) {
    int min = arr[0];
    int pos = 0;

    for (int i = 1; i < n; i++) {
        if (arr[i] < min) {
            min = arr[i];
            pos = i;
        }
    }
    printf("Minimum Element: %d at Position: %d\n", min, pos);
}

```

```

int main() {
    int arr[] = {50, 20, 10, 30, 15};
    int n = sizeof(arr) / sizeof(arr[0]);
    findMinAndPosition(arr, n);
    return 0;
}

```

21. Write a program using function to find the second largest element in an array of n elements.

C Program to Find Second Largest Element

```
#include <stdio.h>
#include <limits.h>

void findSecondLargest(int arr[], int n) {
    int max = INT_MIN;
    int secondMax = INT_MIN;

    for (int i = 0; i < n; i++) {
        if (arr[i] > max) {
            secondMax = max;
            max = arr[i];
        } else if (arr[i] > secondMax && arr[i] != max) {
            secondMax = arr[i];
        }
    }
    if (secondMax == INT_MIN)
        printf("No second largest element found\n");
    else
        printf("Second Largest Element: %d\n", secondMax);
}

int main() {
    int arr[] = {12, 35, 1, 10, 34, 1};
    int n = sizeof(arr) / sizeof(arr[0]);
    findSecondLargest(arr, n);
    return 0;
}
```

22. Write an algorithm to find the minimum, the maximum, and how many times they both occur in an array of n elements.

C Algorithm to Find Min, Max, and Their Frequency

```
#include <stdio.h>

void findMinMaxAndFrequency(int arr[], int n) {
    int min = arr[0], max = arr[0];
```

```

int minCount = 1, maxCount = 1;

for (int i = 1; i < n; i++) {
    if (arr[i] > max) {
        max = arr[i];
        maxCount = 1;
    } else if (arr[i] == max) {
        maxCount++;
    }

    if (arr[i] < min) {
        min = arr[i];
        minCount = 1;
    } else if (arr[i] == min) {
        minCount++;
    }
}

printf("Min: %d, Frequency: %d\n", min, minCount);
printf("Max: %d, Frequency: %d\n", max, maxCount);
}

int main() {
    int arr[] = {12, 12, 45, 23, 45, 12, 45};
    int n = sizeof(arr) / sizeof(arr[0]);
    findMinMaxAndFrequency(arr, n);
    return 0;
}

```

23. Remove all duplicates from an ordered array and contract the array accordingly.

C Program to Remove Duplicates from an Ordered Array

```
#include <stdio.h>
```

```

int removeDuplicates(int arr[], int n) {
    if (n == 0 || n == 1)
        return n;

    int j = 0;
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] != arr[i + 1]) {

```

```

        arr[j++] = arr[i];
    }
}
arr[j++] = arr[n - 1];
return j;
}

int main() {
    int arr[] = {10, 10, 20, 30, 30, 30, 40};
    int n = sizeof(arr) / sizeof(arr[0]);
    n = removeDuplicates(arr, n);
    printf("Array after removing duplicates: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}

```

24. Write a program to delete from an ordered array all elements that occur more than k times.

```
#include <stdio.h>
```

```
// Function to delete elements that occur more than k times
```

```
int deleteMoreThanK(int arr[], int n, int k) {
    int i, j = 0, count;

    for (i = 0; i < n; ) {
        count = 1;
        // Count occurrences of the current element
        while (i + 1 < n && arr[i] == arr[i + 1]) {
            count++;
            i++;
        }
    }
}

```

```
// Include element only if its occurrences are <= k
```

```
if (count <= k) {
    for (int x = 0; x < count; x++) {
        arr[j++] = arr[i];
    }
}

```



```

    }
    i++;
}
return j; // Return new size of array
}

// Display Array Elements
void display(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {10, 10, 20, 20, 20, 30, 30, 40, 40, 40, 40};
    int n = sizeof(arr) / sizeof(arr[0]);
    int k = 2;

    printf("Original Array: ");
    display(arr, n);

    n = deleteMoreThanK(arr, n, k);
    printf("Modified Array (After Removing Elements > k times): ");
    display(arr, n);

    return 0;
}

```

25. Write a program to partition the elements of an array into two subsets such that elements less than or equal to x are in one subset and elements greater than x are in another subset.

```
#include <stdio.h>
```

```

// Function to Partition the Array
void partitionArray(int arr[], int n, int x) {
    int low = 0, high = n - 1;

    while (low <= high) {
        // Move low forward if element is <= x

```

```

    while (arr[low] <= x) {
        low++;
    }
    // Move high backward if element is > x
    while (arr[high] > x) {
        high--;
    }
    // Swap elements to maintain partition
    if (low < high) {
        int temp = arr[low];
        arr[low] = arr[high];
        arr[high] = temp;
    }
}

// Display Function
void display(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {12, 7, 14, 9, 10, 6, 18, 20, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 10;

    printf("Original Array: ");
    display(arr, n);

    partitionArray(arr, n, x);

    printf("Partitioned Array (<= %d on Left, > %d on Right): ", x, x);
    display(arr, n);

    return 0;
}

```

}

26. Consider the linear arrays AAA[5:50], BBB[-5:10], and CCC[1:8].

(a) Find the number of elements in each array.

To calculate the number of elements in a linear array, use the formula:

$$\text{Number of Elements} = \text{Upper Bound} - \text{Lower Bound} + 1$$

1.

AAA[5:50]

$50 - 5 + 1 = 46$ elements

2. **BBB[-5:10]**

$10 - (-5) + 1 = 16$ elements

3. **CCC[1:8]**

$8 - 1 + 1 = 8$ elements

(b) Suppose Base(AAA) = 300 and $w = 4$ words per memory cell for AAA. Find the address of AAA[15], AAA[35], and AAA[55].

Address Calculation Formula (Row-Major Order)

$$\text{Address} = \text{Base Address} + (\text{Index} - \text{Lower Bound}) \times \text{Word Size}$$

•

Base Address = 300

• Word Size = 4 Words

• Lower Bound of AAA = 5

1. **Address of AAA[15]**

$\text{Address} = 300 + (15 - 5) \times 4 = 340$

2. **Address of AAA[35]**

$\text{Address} = 300 + (35 - 5) \times 4 = 420$

3. **Address of AAA[55] (*Invalid Calculation as Index 55 is Out of Bounds*)**

Error: Index Out of Range. The valid range is from 5 to 50. \text{Error: Index Out of Range. The valid range is from 5 to 50.}Error: Index Out of Range. The valid range is from 5 to 50.

27. Multidimensional Arrays P and Q

- $P[-2:2, 2:22]$
- $Q[1:8, -5:5, -10:5]$

(a) Find the length of each dimension.

For a dimension:

$$\text{Length} = \text{Upper Bound} - \text{Lower Bound} + 1$$

1. **Array P**

- First Dimension: $2 - (-2) + 1 = 5$
- Second Dimension: $22 - 2 + 1 = 21$

2. **Array Q**

- First Dimension: $8 - 1 + 1 = 8$
- Second Dimension: $5 - (-5) + 1 = 11$
- Third Dimension: $5 - (-10) + 1 = 16$

(b) Find the number of elements in P and Q.

(b) Find the number of elements in P and Q.

Total Elements = Length of Dimension 1 $\times$ Length of Dimension 2 $\times$ Length of Dimension 3 (if applicable)

1. **Array P:**

$$5 \times 21 = 105 \text{ elements}$$

2. **Array Q:**

$$8 \times 11 \times 16 = 1408 \text{ elements}$$

(c) Find the Effective Indices and Address of Q[3, 3, 3].

(c) Find the Effective Indices and Address of Q[3, 3, 3].

Effective Index Formula (For Column-Major Order)

$$E = (k - L_3) \times (N_1 \times N_2) + (j - L_2) \times N_1 + (i - L_1)$$

- Base Address (Q) = 400
- Word Size (W) = 4
- Indices: $i = 3, j = 3, k = 3$
- Lower Bounds: $L_1 = 1, L_2 = -5, L_3 = -10$
- Dimensions: $N_1 = 8, N_2 = 11, N_3 = 16$ 

Effective Indices Calculation

$$E = (3 - (-10)) \times (8 \times 11) + (3 - (-5)) \times 8 + (3 - 1)$$

$$E = (3 + 10) \times 88 + (3 + 5) \times 8 + (3 - 1)$$

$$E = 13 \times 88 + 8 \times 8 + 2$$

$$E = 1144 + 64 + 2 = 1210$$

Address Calculation

$$\text{Address} = \text{Base Address} + E \times \text{Word Size}$$

$$\text{Address} = 400 + 1210 \times 4$$

$$\text{Address} = 400 + 4840 = 5240$$